你的栈到底有多大：爆栈与任务栈计算

嵌入式 Linux 手记　2026-10-29　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。这段失控递归，分别在两个平台上跑会怎样?

```
void recurse(int n) {
    char pad[4096];       /* 每层多占 4KB */
    recurse(n + 1);       /* 没有出口 */
}
```

在 Linux 上（以默认 `-O0` 编译；若开 `-O2` 以上优化，这个尾递归会被改写成循环，永远跑不爆），它轰轰烈烈地死：栈一路生长触到上限，触发段错误，`Segmentation fault` 明明白白。在一块 Cortex-M 单片机上，它可能什么提示都没有：栈指针悄悄越过边界，把相邻内存里的数据一个一个覆盖掉，别的任务的上下文、队列句柄、配置变量，谁住得近谁遭殃。程序继续跑，只是开始"偶尔抽风"，而且每次抽风的位置都不一样。同一个 bug，两种死相，差距全在"有没有人对栈设防"。这一篇把栈的账本量到底：上限从哪来、爆栈怎么个死法、需求怎么算、三道闸怎么设。

一、三个世界，三种栈上限

栈不是无限资源，上限由所在平台规定，而且差距悬殊。

Linux 主线程的栈上限默认 8MB（`ulimit -s` 可查可改，内核侧对应 `RLIMIT_STACK`）。注意篇 16 讲过的按需调页在这里同样生效：8MB 是上限而非实占，栈页按需逐页挂进页表，触碰哪页分配哪页，所以"程序没吃满 8MB"不代表它申请不到 8MB。线程的栈在创建时一次性 `mmap`，glibc 默认大小跟随 `ulimit -s`，需要更小/更大时用 `pthread_attr_setstacksize` 显式指定。

内核栈的数字要小三个数量级：x86-64 上通常只有 16KB。驱动开发由此有一条铁律——内核态禁大局部数组、禁深递归——你写的每一层函数调用、每一个 `char buf[4096]`，都在跟中断处理共享这 16KB。用户态"随手开个大数组"的习惯，搬进内核就是爆栈事故。

RTOS 世界更紧：每个任务的栈在创建时静态指定，典型任务几百字节到几 KB。TCP/IP 协议栈任务或跑 printf 的任务可能给到 2~4KB，一个只收消息的小任务 128 字节也有。为什么抠成这样？单片机的总 RAM 本身往往只有几十 KB，栈是按任务数量倍乘的开销，8MB 的奢侈想都不用想。

世界	典型上限	上限来源
Linux 主线程	8MB（默认）	`ulimit -s` / `RLIMIT_STACK`
Linux 内核	8~16KB	架构定义（x86-64 为 16KB）
RTOS 单任务	128B ~ 几 KB	创建任务时静态指定



图：三个世界三种栈上限。数量级差距一目了然；Linux 的 8MB 是上限不是实占，栈页按需挂进页表。

二、爆栈的两种死相

栈写穿边界后发生什么，取决于有没有人设防。

有设防的世界死得明白。Linux 主线程的栈区带守护页（guard page），越界写入落在未映射页上，MMU 翻译失败，走篇 17 那条三层判决流水线，SIGSEGV 现场处决；`-fstack-protector` 还在每帧安放 canary 金丝雀值，函数尾声发现被改写立即调 `__stack_chk_fail` 终止进程。带 MPU 的 MCU 也能做到类似效果：把任务栈边界设为不可访问，越界立刻触发 fault。崩溃难看，但死在案发现场。

没有设防的世界死得无声。Cortex-M 裸机或未配 MPU 的 RTOS 上，栈指针越界没有任何硬件拦截，写入继续进行，覆盖的是恰好住在栈旁边的对象：别的任务的控制块、全局变量、队列存储区。更糟的是这些对象被破坏的时机完全随机，症状随之漂移：今天崩溃在 A 任务，明天串口输出乱码，后天配置莫名归零。这类"偶发怪病"是嵌入式调试的顶级噩梦，真凶经常就是一个算小了的任务栈。记住这个对比：栈溢出在带保护系统里是段错误，在裸系统里是慢性病。

三、栈需求怎么算：静态估算与动态水位

算栈需求有两条互补的路。

静态估算走编译器。GCC 的 `-fstack-usage` 让每个函数生成一个 `.su` 文件，记录本函数一帧的字节数；配 `-Wstack-usage=N` 可以对超过 N 字节的函数直接告警。单函数帧大小有了，再沿调用图找最深路径逐层求和，就是最坏情况的静态栈深。递归是这条路的天敌（深度不定），遇到只能按最大深度乘每层帧大小估算，或干脆禁止递归。RTOS 工程里，`-fstack-usage` 审计 + 红线告警是入职新项目的常规体检。

动态测量走水位。FreeRTOS 创建任务时用 `0xA5` 填满任务栈，运行中被覆盖的区域就是真实触达过的部分；`uxTaskGetStackHighWaterMark()` 返回历史最低剩余量（单位为栈字 `StackType_t`），"水位"越低说明栈越接近用满。工程惯例是让满载跑够长时间，水位逼近 0 即爆栈前兆；定任务栈大小时在观测峰值上留 30%~50% 的裕量（经验值，随任务关键度加码）。Linux 侧另有 `ulimit -s` 的上限概念，但那是配额不是用量，实际用量要靠栈回溯或插桩统计。

两法合用：静态估算给出设计值（写进任务创建代码），动态水位给出验证值（量产前的测试项），缺一不可。

四、防爆三道闸

第一道闸，静态闸：设计期就把风险拒之门外。递归要么设深度上限、要么改迭代；大局部数组（比如那个 `char buf[8*1024*1024]`）改静态分配或走堆；`-Wstack-usage` 设红线，超标函数过不了编译审查。每个新增函数在 `.su` 文件里留痕，栈账目随代码一起进版本管理。

第二道闸，运行闸：让越界在第一时间被发现。FreeRTOS 打开 `configCHECK_FOR_STACK_OVERFLOW=2`，同时做两项校验：栈指针是否越界 + 栈底 `0xA5` 填充图案是否被破坏，一旦确认溢出进入 `vApplicationStackOverflowHook()` 钩子，你可以在这里记录任务名、打点、复位，把"随机抽风"变成"确定性报警"。桌面侧的等价物是 `-fstack-protector` 的 canary。

第三道闸，硬件闸：把边界交给 MPU。带 MPU 的 MCU 可以把每个任务栈的边界区域设为不可访问，越界访问当场触发 fault，进入硬错误处理，这是裸机世界能拿到的最强防线，等于给每个任务配上了 Linux 主线程才有的守护页。三道闸从便宜到贵、从设计期到运行期，按项目资源依次叠上。



图：爆栈的两种死相与防爆三道闸。静态闸在设计期拒险，运行闸让越界确定性报警，硬件闸把边界交给 MPU。

写在最后

1. 栈上限三个世界三档数字：Linux 主线程默认 8MB（`ulimit -s`，栈页按需增长）、内核栈 8~16KB（禁大数组禁深递归是铁律）、RTOS 单任务几百字节（创建时静态指定）。
2. 爆栈两种死相：有守护页/canary/MPU 的系统死得明白（SIGSEGV/fault），无保护的系统悄悄写穿相邻对象，症状随机漂移——嵌入式偶发怪病的常见真凶。
3. 需求计算两路合用：`-fstack-usage` 静态估算最深调用路径 + `uxTaskGetStackHighWaterMark()` 动态水位验证；定栈大小在峰值上留 30%~50% 裕量（经验值）。
4. 防爆三道闸：静态闸（红线告警、禁递归、大数组搬家）、运行闸（`configCHECK_FOR_STACK_OVERFLOW=2` 双重校验进钩子）、硬件闸（MPU 边界不可访问）。
5. 递归是静态估算的天敌，深度不定的递归在 RTOS 上等于不定时炸弹。

栈的极限量完了，下一个主角是堆。`malloc` 返回的指针看起来天下无敌，可程序越跑越占内存、重启才释放的怪象层出不穷。下一篇《malloc 之后内存去哪了：为何程序越跑越占内存》，进 glibc 的批发仓库看 chunk、arena 与 RSS 只涨不降的真相。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。